

The Definitive Technical Guide to Forking Olympus DAO and Designing IDC v3 (Inheritable Debt Currency)

TL;DR

- The correct fork target is the `OlympusDAO/olympus-v3` (Bophades) repo on `master`, not the legacy `olympus-contracts` Hardhat repo — Bophades is the canonical Foundry-based, Default-Framework codebase whose live mainnet Kernel is `0x2286d7f9639e8158FaD1169e76d1FbC38247f54b`, and whose monetary policy stack (MINTR/TRSRY/RANGE/PRICE/ROLES/DLGTE/DEPOS modules + Operator/Heart/MonoCooler/EmissionManager policies) is exactly the pattern IDC v3 should rename and re-deploy.
- IDC v3 should keep Bophades's Kernel/Modules/Policies layout verbatim, but replace OHM with an IDC token backed by a basket of {iDEBT, BTC (via WBTC/cbBTC), USDS, GHO, aEthUSDC}; replace USDS in Cooler with a chosen reserve; and add three new modules — DEBT (iDEBT position registry), AAVE (yield router into Aave V3 Pool `0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2`), and HEIR (inheritance/successor logic on Cooler positions). All policies remain governance-rotatable via Kernel `Actions`. `Aave Governance`
- **Critical licensing constraint:** the olympus-v3 sources are SPDX-tagged `AGPL-3.0-or-later` per-file (gOHM is verified on Etherscan under "GNU AGPLv3 license"), so a Bophades fork is itself AGPL-3.0; you cannot relicense it Apache-2.0. Write all new IDC v3 code (DEBT, HEIR, AAVE modules; IDC token; deploy scripts; mindX/BANKON glue) as Apache-2.0 in a separate `idc-core` package and *import* the AGPL Kernel/Modules as a `lib/` submodule — the combined deployment is AGPL, but your new contributions stay Apache-2.0 licensable. `Etherscan`

Key Findings

1. **Bophades (olympus-v3) is the only fork target worth touching.** It is Foundry-native, modular, audited (Code4rena 2022-08), and continuously upgraded — MonoCooler V2, EmissionManager v1.2, Convertible Deposits v1.0, Umbrella-compatible safety patterns all live in the same repo.
2. **Cooler V2's "0.5% APR, no price liquidation, perpetual" model is the single most important monetary innovation to inherit.** Per Olympus docs: *"Loans have an annualized interest rate of 0.5%, as approved by OCG Proposal 8"*; defaults only trigger when unpaid interest exceeds a governance-defined threshold; the LTV Drip Rate (`0.0000011574`)

`USDS/second = 0.1 USDS/day`) is a separate origination-LTV parameter, not the interest rate. [\(Olympusdao\)](#)

3. **The Default Framework's Kernel Actions** (`InstallModule`, `UpgradeModule`, `ActivatePolicy`, `DeactivatePolicy`, `ChangeExecutor`, `MigrateKernel`) give you a single chokepoint for governance, perfect for swapping policies (e.g. swap RBS Operator from v1.5 to a IDC-tuned v2 without redeploying TRSRY). [\(Etherscan\)](#)
 4. **Aave V3 integration is straightforward via the address book.** [\(aave-address-book\)](#) exports `AaveV3Ethereum.POOL == 0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2`. The on-chain `PoolAddressesProvider` (`0x2f39d218133AFaB8F2B819B1066c7E434Ad94E9e`) is the canonical registry — always query it rather than hard-coding. [\(GitHub\)](#) [\(Aave Governance\)](#)
 5. **stkGHO is the safest yield rail in Aave today** — 0% slashing per Aave's official Umbrella docs (verbatim from [\(aave.com/help/umbrella/stake\)](#): "*stkGHO: Maximum slashing risk is up to 0% of the staked assets (slashing disabled)*"), while legacy stkAAVE/stkABPT retain "*up to 20% of the staked assets.*" This makes stkGHO an attractive component of an IDC SAFE module. [\(Aave\)](#) [\(Aave\)](#)
 6. **The Inheritable-Debt mechanism has no existing analog in DeFi.** Even Cooler V2's DLGTE is only a delegation primitive, not a successor primitive. The `HEIR_` module is genuinely novel and is the strongest product differentiator IDC v3 can offer.
-

Details

Part 1 — Definitive Guide to Forking Olympus DAO

1.1 Evolution: V1 → V2 → V3 (Bophades) → Cooler V2 + Emissions Manager + Convertible Deposits

Olympus V1 (March 2021). Per IQ.wiki: "*OHM was launched in March 2021 through a Discord offering and an Initial Decentralized Exchange (DEX) Offering (IDO)*"; CoinMarketCap dates the trading launch to March 23, 2021. Hardhat-based. Core: `OlympusERC20Token` (OHM, `0x383518188c0c6d7730d91b2c03a03c837814a899`), `sOHM V1` (`0x04F2694C8fcee23e8Fd0dfEA1d4f5Bb8c352111F`), `wsOHM V1` (`0xCa76543Cf381ebBB277bE79574059e32108e3E65`), `V1 Treasury` (`0x31F8Cc382c9898b273eff4e0b7626a6987C846E8`), `V1 Staking` (`0xFd31c7d00Ca47653c6Ce64Af53c1571f9C36566a`), `V1 Distributor` (`0xC58E923bf8A00E4361FE3f4275226a543D7D3ce6`), and a swarm of bond depositories (DAI, FRAX, LUSD, ETH, OHM-DAI LP). Rebase mechanism: per-epoch supply expansion to stakers; APY was set by `_rewardRate` on the Distributor (originally `5700` bps over `2200`-block epochs at construction). The (3,3) game theory: stake-stake is the Nash optimum because both bond-and-stake compound exponentially while only sell-sell is value-destructive. **Do not fork V1.** It is Hardhat, monolithic, and superseded. [\(IQ.wiki\)](#) [\(Etherscan\)](#)

Olympus V2 (December 14, 2021). Per the OlympusDAO migration page (docs.olympusdao.finance/main/basics/migration): "You have two months to migrate after V2 launch (14th December 2021)", corroborated by the OlympusDAO Medium post "Get Ready for Olympus V2 Migration" dated December 23, 2021. Introduced (gOHM) ((0xab87046fBb341D058F17CBC4c1133F25a20a52f), non-rebasing index-tracked wrapper), (OHM V2) ((0x64aa3364F17a4D01c6f1751Fd97C2BD3D7e7f1D5)), (sOHM V2) ((0x04906695D6D12CF5459975d7C3C03356E4Ccd460)), Olympus Authority ((0x1c21F8EA7e39E2BA00BC12d2968D63F4acb38b7A)), (Staking V2) ((0xB63cac384247597756545b500253ff8E607a8020)), (Treasury V2) ((0x9A315BdF513367C0377FB36545857d12e85813Ef)), (BondDepositoryV2) ((0x9025046c6fb25Fb39e720d97a8FD881ED69a1Ef6)), (YieldDirector) ((0x2604170762A1dD22BB4F96C963043Cd4FC358f18)). V2 was still ownership-based (IOlympusAuthority), not modular. (Ticker Report + 2)

Olympus V3 / Bophades (audited Code4rena Aug-2022, deployed shortly after). This is what you fork. It moves the protocol onto the **Default Framework** — a modular (Kernel + Modules + Policies) architecture where each Module is a 5-byte (KEYCODE) (e.g. "TRSRY", "MINTR", "PRICE", "RANGE", "ROLES", "DLGTE", "DEPOS"), and Policies are stateless-ish facades that declare (configureDependencies()) returning (Keycode[]) and (requestPermissions()) returning ((Keycode, bytes4 funcSelector)[]). The Kernel exposes six (Actions): (InstallModule, UpgradeModule, ActivatePolicy, DeactivatePolicy, ChangeExecutor, MigrateKernel). The current mainnet executor is the DAO multisig ((0x245cc372C84B3645Bf0Ffe6538620B04a217988B)) (V1Migrator, governance proposals execute via (Timelock) ((0x953EA3223d2dd3c1A91E9D6cca1bf7Af162C9c39)) and (GovernorBravoDelegator) ((0x0941233c964e7d7Efeb05D253176E5E634cEFfcD))). (GitHub +2 +3)

Range Bound Stability (RBS). A policy stack — (Operator.sol) (current v1.5: ((0x6417F206a0a6628Da136C0Faa39026d0134D2b52))), (Heart.sol) (current v1.7: ((0x5824850D8A6E46a473445a5AF214C7EbD46c5ECB))), (BondCallback.sol) ((0x73df08CE9dcC8d74d22F23282c4d49F13b4c795E)), (PriceConfig.sol) ((0xf6D5d06A4e8e6904E4360108749C177692F59E90)) — that uses (PRICE) module historical observations to compute a moving-average target price for OHM and constructs four guardrails: (GitHub)

- (Lower Wall = MA × (1 - Wall Spread))
- (Lower Cushion = MA × (1 - Cushion Spread))
- (Upper Cushion = MA × (1 + Cushion Spread))
- (Upper Wall = MA × (1 + Wall Spread)) with (Cushion Spread < Wall Spread).

(Olympusdao)

Inside the cushion bands, RBS deploys Bond Protocol bond markets (SDA — Sequential Dutch Auctions) to buy/sell OHM against reserves at a discount. At the walls, RBS swaps directly with the Treasury, capacity equal to a configured $\text{bid factor} \times \text{treasury reserves}$. After wall depletion, regeneration requires price to remain outside MA for X of last Y epochs (governance parameters), plus a minimum cooldown. $\text{Forgd} + 2$

Cooler Loans V1 (OIP-144). Three contracts: `Cooler.sol` (escrow), `CoolerFactory.sol` (clones-with-immutable-args factory, `0x30Ce56e80aA96EbbA1E1a74bC5c0FEB5B0dB4216`), `Clearinghouse.sol` (the lender; current V1 is deprecated at `0x1e094fE00E13Fd06D64EeA4FB3cD912893606fE0` and `0xE6343ad0675C9b8D3f32679ae6aDbA0766A2ab4c` — the legacy v1.1 — and the original `0xD6A6E8d9e82534bD65821142fcCd91ec9cF31880`). The Clearinghouse is a Kernel Policy with permissions on `TRSR` and `MINTR`; idle DAI was held as `sDAI` (Maker DSR) for yield. [GitHub](#)

Cooler Loans V2 / MonoCooler (deployed 2025). Replaces the entire fixed-term, expiring V1 model with a single perpetual position per user. Mainnet contracts: [Olympusdao](#)

- `MonoCooler` policy: `0xdb591Ea2e5Db886dA872654D58f6cc584b68e7cC`
`mcp__Blockscout__get_ad...`
- `Cooler V2 LTV Oracle`: `0x9ee9f0c2e91E4f6B195B988a9e6e19efcf91e8dc`
- `Cooler V2 Treasury Borrower`: `0xD58d7406E9CE34c90cf849Fc3eed3764EB3779B0`
- `DLGTE` module (delegation registry for Cooled gOHM):
`0xD3204Ae00d6599Ba6e182c6D640A79d76CdAad74`
- Reserve token (debt): **USDS** (Sky) `0xdC035D45d973E3EC169d2276DDab16f1e407384F`
- Collateral: gOHM `0x0ab87046fBb341D058F17CBC4c1133F25a20a52f`

Per the Olympus docs: "Loans have an annualized interest rate of 0.5%, as approved by OCG Proposal 8". The LTV Drip Rate is a separate parameter — "max (positive) rate of change of Origination LTV allowed: 0.0000011574 USDS/second (0.1 USDS/day)" — that increases the origination LTV linearly toward a governance-set target (initial origination LTV May 15, 2025: `2961.64 USDS/gOHM`, Liquidation Premium 1%). Defaults trigger only when unpaid interest exceeds a governance-defined threshold (no price-based liquidation). `DLGTE` clones a `DelegateEscrow` per delegate; up to 10 delegates per position. [Olympusdao](#) [Olympusdao](#)

Emissions Manager (current v1.2: `0xA61b846D5D8b757e3d541E0e4F80390E28f0B6Ff`). Sells OHM into the market when $\text{market price} / \text{backing} > \text{min premium}$. $\text{Current Emission Rate} = \text{Base Rate} \times (1 + \text{premium}) / (1 + \text{min premium})$; runs three times daily on the Heart heartbeat. Backed-by-bond shortfalls trigger a bond market via Bond Protocol. [Olympusdao](#)

Yield Repurchase Facility (`0x271e35a8555a62f6bA76508E85dFD76D580B0692`). Calculates weekly treasury yield (mostly sUSDS/sUSDe), buys back OHM over the following 7 days price-agnostic. `Travis Wiedower`

Convertible Deposits (OIP-186 / v1.0 policies deployed 2025). `ConvertibleDepositFacility`
`0xEBDe552D851DD6Dfd3D360C596D3F4aF6e5F9678`, `ConvertibleDepositAuctioneer`
`0xF35193DA8C10e44aF10853Ba5a3a1a6F7529E39a`,
`ConvertibleDepositAuctioneerLimitOrders`
`0x7d8f82A0D5B67d5FDd1B77A899FF517818FaFc2e`, `DepositManager`
`0xcb4E21Eb404d80F3e1dB781aAd9AD6A1217fbbf2`, `DepositRedemptionVault`
`0x20a3d8510f2e1176E8Db4CeA9883a8287a9029Db`, `DEPOS` module
`0x02331A4c97a4841084dF54d7c0eC04DD3f1A9F1c`. Receipts are ERC-6909 (optionally wrappable to ERC-721). `Olympusdao`

1.2 Authoritative on-chain registry (Ethereum mainnet, source: docs.olympusdao.finance/main/contracts/addresses)

Multisigs

Role	Address
DAO multisig	<code>0x245cc372C84B3645Bf0Ffe6538620B04a217988B</code>
Emergency multisig	<code>0xa8A6ff2606b24F61AFA986381D8991DFcCCd2D55</code>
Policy multisig (deprecated)	<code>0x0cf30dc0d48604A301dF8010cdc028C055336b2E</code>

Bophades core

Component	Address
Kernel	0x2286d7f9639e8158FaD1169e76d1FbC38247f54b
TRSRY (v1.0)	0xa8687A15D4BE32CC8F0a8a7B9704a4C3993D9613
MINTR (v1.0)	0xa90bFe53217da78D900749eb6Ef513ee5b6a491e
PRICE (v1.1)	0xd6C4D723fdadCf0D171eF9A2a3Bfa870675b282f
RANGE (v2.0)	0x399cD3685912bb56aAeD0949119dB6cE5Df60FB5
ROLES (v1.0)	0x6CAfd730Dc199Df73C16420C4fCAb18E3afbFA59
BLREG (v1.0)	0x375E06C694B5E50aF8be8FB03495A612eA3e2275
DLGTE (v1.0)	0xD3204Ae00d6599Ba6e182c6D640A79d76CdAad74
DEPOS (v1.0)	0x02331A4c97a4841084dF54d7c0eC04DD3f1A9F1c

Active policies (selected)

Policy	Address
Operator (v1.5)	0x6417F206a0a6628Da136C0Faa39026d0134D2b52
Heart (v1.7)	0x5824850D8A6E46a473445a5AF214C7EbD46c5ECB
BondCallback	0x73df08CE9dcC8d74d22F23282c4d49F13b4c795E
EmissionManager (v1.2)	0xA61b846D5D8b757e3d541E0e4F80390E28f0B6Ff
YieldRepurchaseFacility (v1.2)	0x271e35a8555a62F6bA76508E85dfD76D580B0692
MonoCooler (Cooler V2)	0xdb591Ea2e5Db886dA872654D58f6cc584b68e7cC
Cooler V2 LTV Oracle	0x9ee9f0c2e91E4f6B195B988a9e6e19efcf91e8dc
Cooler V2 Treasury Borrower	0xD58d7406E9CE34c90cf849Fc3eed3764EB3779B0
RolesAdmin	0xb216d714d91eeC4F7120a732c11428857C659eC8
TreasuryCustodian	0xC9518AC915e46D707585116451Dc19c164513Ccf
Emergency	0x9229b0b6FA4A58D67Eb465567DaA2c6A34714A75
Burner	0x9f08c2603e919a46d6D98289C9ADA5250b310558
CrossChainBridge (LayerZero)	0x45e563c39cddb8699a90078f42353a57509543a
CCIPCrossChainBridge	0xFbf6383dC3F6010d403Ecdf12DDC1311701D143D

Tokens

Token	Address
OHM V2	0x64aa3364F17a4D01c6f1751Fd97C2BD3D7e7f1D5
sOHM V2	0x04906695D6D12CF5459975d7C3C03356E4Ccd460
gOHM	0x0ab87046fBb341D058F17CBC4c1133F25a20a52f
OHM V1 (legacy)	0x383518188c0c6d7730d91b2c03a03c837814a899

1.3 GitHub repositories (canonical, current as of May 2026)

Repo	Purpose	Notes
github.com/OlympusDAO/olympus-v3	Bophades — Foundry, master branch GitHub GitHub	Fork target. SPDX per-file AGPL-3.0-1 src/modules/{MINTR, TRSRY, PRIC src/policies/*
github.com/OlympusDAO/olympus-contracts	V1/V2 Hardhat GitHub	Legacy reference only
github.com/OlympusDAO/olympus-docs	Docs source	Useful for spec drift
github.com/ohmzeus/Cooler	Cooler V1 reference	Walkthrough: ag0.gitbook.io/cool
github.com/OlympusDAO/olympus-frontend	TS dApp GitHub	Reference for sub-graph queries
github.com/OlympusDAO/{bonds,rbs,cooler-loans,convertible-deposits,staking}-subgraph	TheGraph indexers	Useful for IDC analytics
github.com/code-423n4/2022-08-olympus	Public audit scope	Useful for invariant lists

1.4 Forking strategy with Foundry (production-grade)

Project layout (flat, per cypherpunk2048 conventions):


```

idc-v3/
├─ foundry.toml
├─ remappings.txt
├─ lib/
│   ├─ olympus-v3/          # git submodule, AGPL – read-only reference
│   ├─ solmate/             # transmissions11/solmate, MIT
│   ├─ solady/              # Vectorized/solady, MIT
│   ├─ openzeppelin-contracts/ # MIT
│   ├─ forge-std/           # MIT
│   ├─ aave-address-book/    # BUSL-1.1 import-only, addresses
│   └─ aave-v3-core/         # BUSL-1.1 reference only
├─ src/
│   └─ Kernel.sol           # Apache-2.0 – clean-room rewrite OR vendored AGPL
fork
│   ├─ IDC.sol              # Apache-2.0 – new ERC20Permit (Solmate base)
│   ├─ sIDC.sol
│   ├─ gIDC.sol
│   └─ modules/
│       ├─ TRSRY/           # KEYCODE "TRSRY"
│       ├─ MINTR/           # KEYCODE "MINTR"
│       ├─ PRICE/           # KEYCODE "PRICE"
│       ├─ RANGE/           # KEYCODE "RANGE"
│       ├─ ROLES/           # KEYCODE "ROLES"
│       ├─ DEBT/            # KEYCODE "DEBT_" – NEW: iDEBT NFT positions
│       ├─ AAVE/            # KEYCODE "AAVEY" – NEW: yield router
│       └─ HEIR/            # KEYCODE "HEIR_" – NEW: inheritance registry
│   └─ policies/
│       ├─ Operator.sol     # RBS for IDC
│       ├─ Heart.sol        # keeper entrypoint
│       ├─ EmissionManager.sol
│       ├─ CoolerV3.sol     # MonoCooler + HEIR hook
│       ├─ AaveAllocator.sol
│       ├─ BankonGate.sol   # BONAFIDE-reputation gate
│       └─ MindXOracle.sol  # mindx.pythai.net API consumer (signed)
├─ script/
│   ├─ 00_DeployKernel.s.sol
│   ├─ 01_DeployModules.s.sol
│   ├─ 02_DeployPolicies.s.sol
│   ├─ 03_ConfigureRoles.s.sol
│   └─ 04_BootstrapTreasury.s.sol
└─ test/
    └─ unit/

```

```
|— integration/  
|— invariant/
```

foundry.toml:

```
toml  
  
[profile.default]  
src          = "src"  
out          = "out"  
libs         = ["lib"]  
solc         = "0.8.25"  
optimizer    = true  
optimizer_runs = 10_000  
via_ir       = true  
fs_permissions = [{ access = "read", path = "./"}]  
ffi          = true  
auto_detect_solc = false  
evm_version   = "cancun"  
gas_reports   = ["*"]  
verbosity    = 3  
  
[fuzz]  
runs = 4096  
  
[invariant]  
runs      = 256  
depth     = 100  
fail_on_revert = false  
  
[rpc_endpoints]  
mainnet  = "${MAINNET_RPC_URL}"  
sepolia  = "${SEPOLIA_RPC_URL}"  
arbitrum = "${ARBITRUM_RPC_URL}"  
base     = "${BASE_RPC_URL}"  
optimism = "${OPTIMISM_RPC_URL}"  
  
[etherscan]  
mainnet = { key = "${ETHERSCAN_KEY}" }
```

remappings.txt:

```
@openzeppelin/=lib/openzeppelin-contracts/  
solmate/=lib/solmate/src/  
solady/=lib/solady/src/  
forge-std/=lib/forge-std/src/  
aave-v3-core/=lib/aave-v3-core/contracts/  
aave-address-book/=lib/aave-address-book/src/  
olympus-v3/=lib/olympus-v3/src/
```

Submodule install:

bash

```
forge init idc-v3 --no-commit  
forge install OlympusDAO/olympus-v3 --no-commit  
forge install transmissions11/solmate --no-commit  
forge install Vectorized/solady --no-commit  
forge install OpenZeppelin/openzeppelin-contracts --no-commit  
forge install aave-dao/aave-address-book --no-commit  
forge install aave/aave-v3-core --no-commit  
forge install foundry-rs/forge-std --no-commit
```

Mainnet-fork test harness pattern (`test/integration/Base.t.sol`):

solidity

```
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.25;

import {Test, console2} from "forge-std/Test.sol";
import {AaveV3Ethereum} from "aave-address-book/AaveV3Ethereum.sol";

abstract contract MainnetForkTest is Test {
    uint256 internal constant FORK_BLOCK = 22_500_000; // pin for determinism

    address internal constant OLYMPUS_KERNEL = 0x2286d7f9639e8158FaD1169e76d1FbC382
    address internal constant OLYMPUS_TRSRY = 0xa8687A15D4BE32CC8F0a8a7B9704a4C399
    address internal constant OLYMPUS_MINTR = 0xa90bFe53217da78D900749eb6Ef513ee5b
    address internal constant OLYMPUS_DAO = 0x245cc372C84B3645Bf0Ffe6538620B04a2
    address internal constant MONO_COOLER = 0xdb591Ea2e5Db886dA872654D58f6cc584b
    address internal constant USDS = 0xdC035D45d973E3EC169d2276DDab16f1e4
    address internal constant GOHM = 0x0ab87046fBb341D058F17CBC4c1133F25a
    address internal constant AAVE_POOL = 0x87870Bca3F3fD6335C3F4ce8392D69350B
    address internal constant AAVE_ORACLE = 0x54586bE62E3c3580375aE3723C14525306
    address internal constant CHAINLINK_ETH = 0x5f4eC3Df9cbd43714FE2740f5E3616155c
    address internal constant CHAINLINK_BTC = 0xF4030086522a5bEEa4988F8cA5B36dbC97
    address internal constant GH0 = 0x40D16FC0246aD3160Ccc09B8D0D3A2cD28

    function setUp() public virtual {
        vm.createSelectFork(vm.rpcUrl("mainnet"), FORK_BLOCK);
        vm.label(OLYMPUS_KERNEL, "Olympus.Kernel");
        vm.label(AAVE_POOL, "Aave.Pool");
        vm.label(MONO_COOLER, "Olympus.MonoCooler");
    }
}
```

1.5 Tokenomics analysis (what to keep, what to drop)

- **Keep:** Treasury-backed value floor, Protocol-Owned Liquidity, RBS, Cooler V2 with no-price liquidation, EmissionManager (only emits when $\text{premium} \geq \text{min premium}$), YRF buybacks, Convertible Deposits as the primary new-issuance path.
- **Drop / replace:** the original (3,3) rebase, Risk-Free Value haircuts as a primary metric (Olympus itself has de-emphasized RFV in favor of "backing per token"), staking-as-yield-mining (use sIDC as a pure non-rebasing index wrapper like gOHM).
- **Yield sources for IDC v3:** (a) Aave V3 deposits via `aEthUSDS` (`0x32a6268f9Ba3642Dda7892aDd74f1D34469A4259`), `aEthUSDC` (`0x98C23E9d8f34FEfb1B7BD6a91B7FF122F4e16F5c`), `aEthDAI` (`0x018008bfb33d285247A21d44E50697654f754e63`); (b) Sky DSR via sUSDS; (c) Cooler V3

interest from IDC borrowers; (d) iDEBT coupon streams; (e) sGHO at 4.25% APR (see Part 2). [Ethplorer](#) [Ethplorer](#)

Part 2 — IDC v3 Scheme Design

2.1 Thesis

Inheritable Debt Currency (IDC) v3 is a Bophades-derived reserve currency whose token ([IDC](#)) is collateralized by a programmable basket: BTC (WBTC/cbBTC) as hard-money numeraire, USDS/DAI/GHO as stable reserves, and iDEBT — a sovereign-debt position NFT collection that tokenizes coupon streams from real-world debt instruments. The protocol *inverts* debt: borrowers extract IDC against gIDC at low fixed interest (Cooler V3), and the protocol holds the *yield-bearing* side (iDEBT coupons + Aave aTokens + DSR), so the system's monetary base earns income while users spend the currency.

The "Inheritable" piece is the killer feature: each Cooler V3 loan is a transferable position that can be willed on-chain to a named successor via the [HEIR](#) module — a deadman-switch ERC-721 inheritance NFT bound to the loan, with BANKON-identity-attested probate.

2.2 Module-by-module specification (KEYCODE → contract)

Keycode	Module	Owns	New vs Olympus
TRSRY	IDCTreasury	Reserve balances; tracks debt allocated to policies	Same shape as Olympus TRSRY
MINTR	IDCMinter	Mint/burn permissions on <code>IDC</code> ERC20	Same
PRICE	IDCPrice	Historical price observations (Chainlink ETH/USD <code>0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419</code> , BTC/USD <code>0xF4030086522a5bEEa4988F8cA5B36dbC97BeE88c</code> , custom Uniswap V3 TWAP oracle for IDC/USDS)	Add BTC feed as TWAP
RANGE	IDCRange	Cushion/wall spreads + capacities	Same
ROLES	IDCRoles	bytes32 role registry + onlyRole modifier consumer	Extended with <code>BONAFIDE_VEL</code> gate
DLGTE	IDCDelegate	gIDC voting-delegation escrows	Identical to Olympus DLGTE
DEPOS	IDCDeposits	Convertible deposit receipts (ERC-6909)	Identical to Olympus DEPOS
DEBT_	IDebtRegistry	NEW ERC-721 collection of iDEBT positions: face value, coupon rate, maturity, issuer rating	Olympus has no
AAVEY	AaveAllocator	NEW module mirroring TRSRY balances as <code>aToken</code> shares; exposes <code>deposit/withdraw/claim</code> callable only by <code>AaveAllocator</code> policy	New
HEIR_	HeirRegistry	NEW maps <code>((positionId) → (heir, deadmanTimeout, lastHeartbeat, socialRecoveryMultisig))</code>	New

Sample DEBT module skeleton (Apache-2.0, Solmate ERC-721 base):

```
solidity
```

```
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.25;

import {Module, Kernel, Keycode, toKeycode} from "src/Kernel.sol";
import {ERC721} from "solmate/tokens/ERC721.sol";

abstract contract DEBTv1 is Module, ERC721 {
    struct Position {
        uint128 faceValue;           // 1e18-scaled
        uint64  couponBps;           // basis points per year
        uint40  maturity;            // unix seconds
        uint8   issuerRating;        // 0=AAA ... 22=D (S&P scale)
        address couponToken;         // payout currency (USDS, USDC, EURS, ...)
    }
    mapping(uint256 => Position) public positions;
    function KEYCODE() public pure override returns (Keycode) { return toKeycode("DEBTv1"); }
    function VERSION() external pure override returns (uint8, uint8) { return (1, 0); }
    function mint(address to, uint256 id, Position calldata p) external virtual permitted;
    function payCoupon(uint256 id, uint256 amount) external virtual permitted;
    function redeem(uint256 id) external virtual permitted returns (uint256);
}
```

2.3 Cooler V3 with HEIR hook

Cooler V3 = MonoCooler(`0xdb591Ea2e5Db886dA872654D58f6cc584b68e7cC`) ported one-to-one with three additions:

1. `bindHeir(uint256 positionId, address heir, uint40 timeoutSeconds)` — owner can name a successor and set a heartbeat-required interval (e.g., 365 days).
2. `heartbeat(uint256 positionId)` — callable by owner OR any delegate registered in DLGTE; resets `lastHeartbeat`.
3. `claimInheritance(uint256 positionId)` — callable by `heir` after `block.timestamp > lastHeartbeat + timeoutSeconds`; transfers Cooler ownership and DLGTE delegation rights atomically. Optional `socialRecoveryMultisig` can override pre-timeout via a 2-of-3 Safe.

This treats the Cooler position as an ERC-721-bound credit estate and integrates directly with BANKON's identity layer (the heir address must be `BONAFIDE_VERIFIED` in the ROLES module).

2.4 AAVE V3 integration

Mainnet addresses (verified by on-chain query against `PoolAddressesProvider`):

Component	Address
Pool (proxy)	<code>0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2</code>
PoolAddressesProvider	<code>0x2f39d218133AFaB8F2B819B1066c7E434Ad94E9e</code>
PoolConfigurator (proxy)	<code>0x64b761D848206f447Fe2dd461b0c635Ec39EbB27</code> <code>Aave Governance</code>
ACLManager	<code>0xc2aaCf6553D20d1e9d78E365AAba8032af9c85b0</code> <code>mcp__Blockscout__read_c...</code>
AaveOracle	<code>0x54586bE62E3c3580375aE3723C145253060Ca0C2</code> <code>mcp__Blockscout__read_c...</code>
AaveProtocolDataProvider (current)	<code>0x0a16f2FCC0D44FaE41cc54e079281D84A363bECD</code> <code>mcp__Blockscout__read_c...</code>
aEthUSDC	<code>0x98C23E9d8f34FEFb1B7BD6a91B7FF122F4e16F5c</code>
aEthDAI	<code>0x018008bfb33d285247A21d44E50697654f754e63</code>
aEthUSDS	<code>0x32a6268f9Ba3642Dda7892aDd74f1D34469A4259</code>
GHO token	<code>0x40D16FC0246aD3160Ccc09B8D0D3A2cD28aE6C2f</code>
variableDebtEthGHO	<code>0x786dBff3f1292ae8F92ea68Cf93c30b34B1ed04B</code> <code>Bitquery</code>
Umbrella stkGHO.v1	<code>0x4f827A63755855cDf3e8f3bcD20265C833f15033</code> <code>Etherscan</code>
Legacy stkGHO (pre-Umbrella)	<code>0x1a88Df1cFe15Af22B3c4c783D4e6F7F9e0C1885d</code> <code>Etherscan</code>

Integration pattern — the `AaveAllocator` policy holds a single `permissioned` function on the `AAVEY` module that calls `IPool.supply(asset, amount, onBehalfOf=AAVEY_module, referralCode=0)` and tracks the aToken share in TRSRY accounting. Withdrawals use `IPool.withdraw(asset, amount, to=TRSRY)`. Rewards are claimed via `RewardsController` periphery. `Aave`

Use the canonical address book to keep code chain-portable:

solidity

```
import {AaveV3Ethereum} from "aave-address-book/AaveV3Ethereum.sol";
// AaveV3Ethereum.POOL == 0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2
```


Aave Safety Module / Umbrella. Per the official Aave Protocol Documentation page (aave.com/docs/developers/safety-module): *"Unlike the legacy Safety Module that required governance intervention for slashing decisions, Umbrella operates through automated smart contract logic. When deficits occur in specific assets, the system immediately burns corresponding staked aTokens to cover the shortfall without manual intervention or governance delays."* Umbrella is deployed on Ethereum supporting USDC/USDT/WETH/GHO. The Aave help page (aave.com/help/umbrella/stake) is the authoritative source for the current slashing parameters and states verbatim: *"stkAAVE and stkABPT: Maximum slashing risk is up to 20% of the staked assets · stkGHO: Maximum slashing risk is up to 0% of the staked assets (slashing disabled)."* Treat stkGHO as a near-risk-free yield component of the IDC SAFE module; build a separate slashing-enabled stkIDC on the UmbrellaStakeToken pattern if you want IDC-denominated insurance. (aave.com) ([Aave](#))

Flashloan use case: IDC will inherit the LoanConsolidator pattern (Olympus uses Maker flashloans today) but switched to Aave V3 ([flashLoan\(\)](#)) / ([flashLoanSimple\(\)](#)) because the Maker DAO Direct Deposit Module has been deprecated post-Sky split. ([Olympusdao](#))

GHO integration option. Per Aave Governance AIP-381, the GHO Stewards may raise the borrow rate up to a hard cap of 9.5% APR in 100 bps increments per 7-day window if the trailing 30-day average GHO price stays outside \$0.995–\$1.005; separately, the sGHO savings vault launched on April 3, 2026 at a fixed 4.25% APR. The IDC TRSRY can borrow GHO by supplying ETH/wstETH and using minted GHO as additional treasury reserves; alternatively park idle GHO in sGHO at 4.25% APR. Borrow interest flows entirely to the Aave DAO — IDC pays for the option of an off-balance-sheet stable liquidity line. ([Eco](#))

2.5 Cross-chain (Wormhole + LayerZero + x402-AVM)

- **Primary settlement:** Ethereum mainnet (canonical IDC supply, all governance).
- **EVM L2s:** Wrapped IDC on Polygon (DAO multisig [0xe06efA3D9Ee6923240ee1195A16ddd96B5CcE8F7](#)), Arbitrum ([0x012BBf0481b97170577745D2167ee14f63E2aD4C](#)), Optimism ([0x559a14a2219Ae81f9a9f857CF31407de2b07F36c](#)), Base, Berachain via the existing Olympus [CrossChainBridge](#) (LayerZero OFT) at [0x45e563c39cddb8699a90078f42353a57509543a](#) and CCIPCrossChainBridge [0xFbf6383dC3F6010d403Ecdf12DDC1311701D143D](#).
- **Algorand bridge:** Wormhole Token Bridge on Ethereum at [0x3ee18B2214AFF97000D974cf647E7C347E8fa585](#) (verified). Forge integration is done by deploying a [CrossChainSender](#) and [CrossChainReceiver](#) per the [wormhole-foundation/wormhole-solidity-sdk](#) patterns. ([Etherscan](#))
- **x402 payments:** [x402-avm](#) (GoPlausible) enables HTTP-402 micropayments for mindX API access using Algorand atomic transaction groups ([@x402-avm/express](#), [@x402-avm/next](#), [@x402-avm/core](#)). The Parsec wallet (a Pera-fork) handles Algorand-side

signing. Recommended pattern: `mindx.pythai.net` exposes `/api/v1/treasury/*` as x402-gated, charging in USDC ASA (Algorand USDC), and the IDC AaveAllocator policy treats inbound x402 receipts as on-chain attestations of mindX-recommended actions.

Goplausible

GitHub

2.6 Ecosystem glue (PYTHAI / DELTAVERSE / BANKON / BONAFIDE / mindX / AgenticPlace)

- **mindX** (`mindx.pythai.net`) — recommends Aave rate-switching, GHO/USDS rotation, RBS spread adjustments. The `MindXOracle` policy is a thin EIP-712-verified consumer: every recommendation is signed off-chain by a mindX key registered in `ROLES`, and a 24-hour timelock applies before any parameter change.
- **AgenticPlace** (`agenticplace.pythai.net`) — a marketplace of `RAGE` agents that can transact via x402; IDC pays them via the Convertible Deposits facility (CD receipts → USDS → x402).
- **BANKON identity** (`bankon.pythai.net`) — supplies KYC-free reputational proofs. Each agent/wallet pair gets a `BONAFIDE_VERIFIED` role in `ROLES` via the `BonaToken / Tessera / Fides` contracts from the BONAFIDE nine-contract suite (`Genius, BonaToken, Tabularium, Fides, SponsioPactum, Censura, Senatus, Tessera, Aerarium/Curia`). Verification gates: Cooler V3 borrow limits, Convertible Deposit auction participation, and HEIR designation.
- **BONAFIDE governance integration** — `Senatus` acts as the on-chain executor for IDC governance proposals; `Aerarium/Curia` is mapped to the TRSRY custodian role; `Censura` is mapped to the Emergency policy `0x9229b0b6FA4A58D67Eb465567DaA2c6A34714A75` analog.
- **DAIO Boardroom** — handles parameter votes (RBS spreads, EmissionManager `base rate`, Cooler V3 LTV target).
- **War Council** — 3-of-5 multisig with permanent `EMERGENCY_SHUTDOWN` role; can pause MonoCooler and AaveAllocator atomically.
- **PYTHAI token (10,000 fixed supply, 40/30/20/10 split)** — `PYTHAI` is the governance ticket for DAIO Boardroom; **PYTHAI does not back IDC**, it gates voting on IDC parameters. This separation prevents the (3,3)-style governance attack where treasury asset doubles as voting asset.

Part 3 — Source-Code Mapping (forkable refs)

3.1 Olympus Bophades core

- Repo: `https://github.com/OlympusDAO/olympus-v3` (branch `master`) `GitHub`

- Paths: `src/Kernel.sol`, `src/modules/MINTR/{MINTR.v1.sol,OlympusMinter.sol}`, `src/modules/TRSRY/{TRSRY.v1.sol,OlympusTreasury.sol}`, `src/modules/PRICE/`, `src/modules/RANGE/`, `src/modules/ROLES/{ROLES.v1.sol,OlympusRoles.sol}`, `src/modules/DLGTE/`, `src/modules/DEPOS/` [GitHub](#) [GitHub](#)
- License per file: `// SPDX-License-Identifier: AGPL-3.0-or-later` (gOHM verified on Etherscan as "GNU AGPLv3 license"). **No top-level LICENSE file** — license is asserted per-file via SPDX headers, so audit each file you vendor.

3.2 Cooler Loans V1 + V2

- V1 reference: [github.com/ohmzeus/Cooler](#) (`Cooler.sol`, `CoolerFactory.sol`, `Clearinghouse.sol`) and walkthrough [ag0.gitbook.io/cooler-loan-code-walkthrough](#)
- V2 (MonoCooler, LoanConsolidator): inside `olympus-v3/src/policies/cooler/` (paths: `MonoCooler.sol`, `LTVOracle.sol`, `TreasuryBorrower.sol`, `LoanConsolidator.sol`, `Composites.sol`)
- Mainnet verified: MonoCooler `0xdb591Ea2e5Db886dA872654D58f6cc584b68e7cC`

3.3 Range Bound Stability

- Code: `olympus-v3/src/policies/Operator.sol`, `olympus-v3/src/policies/Heart.sol`, `olympus-v3/src/policies/BondCallback.sol`, `olympus-v3/src/modules/RANGE/`, `olympus-v3/src/modules/PRICE/`
- Bond Protocol dependency: [github.com/Bond-Protocol/v1-core](#) (`BondFixedTermSDA`, `BondFixedTermTeller`, `BondAggregator`). BondTeller mainnet `0x007F7735baF391e207E3aA380bb53c4Bd9a5Fed6`.

3.4 Aave V3 core + periphery

- Core: [github.com/aave/aave-v3-core](#) (`contracts/protocol/pool/Pool.sol`, `PoolConfigurator.sol`, `tokenization/AToken.sol`, `tokenization/VariableDebtToken.sol`, `libraries/logic/SupplyLogic.sol`, `BorrowLogic.sol`, `FlashLoanLogic.sol`) [Aave](#)
- Periphery: [github.com/aave/aave-v3-periphery](#) (`contracts/rewards/RewardsController.sol`, `UiPoolDataProviderV3.sol`)
- Address book: [github.com/aave-dao/aave-address-book](#) (`src/AaveV3Ethereum.sol` constant `POOL == 0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2`)
- License: `BUSL-1.1`. **Important:** you may *call* Aave contracts freely from any-licensed code; you may not *re-deploy* modified Aave-V3-core sources commercially before BUSL expiry. IDC v3 only *integrates*, so the constraint is satisfied.

3.5 GHO + Umbrella

- `github.com/aave/gho-core` (`contracts/gho/GhoToken.sol`, `facilitators/aave/GhoAToken.sol`, `facilitators/aave/GhoVariableDebtToken.sol`)
- Umbrella: `github.com/aave-dao/aave-umbrella` — `UmbrellaStakeToken` pattern, mainnet stkGHO.v1 `0x4f827A63755855cDf3e8f3bcD20265C833f15033` (EIP-1967 proxy, impl `0x75e8ac0c063b6966e2a9954adedf39bde9370197`)).

3.6 Wormhole

- `github.com/wormhole-foundation/wormhole` (`ethereum/contracts/bridge/Bridge.sol`, `ethereum/contracts/Implementation.sol`)
- Mainnet Token Bridge: `0x3ee18B2214AFF97000D974cf647E7C347E8fa585`
- Solidity SDK: `github.com/wormhole-foundation/wormhole-solidity-sdk`

3.7 Chainlink price feeds (Ethereum mainnet)

- ETH/USD aggregator proxy: `0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419` (8 decimals, heartbeat ~1 h)
 - BTC/USD aggregator proxy: `0xF4030086522a5bEEa4988F8cA5B36dbC97BeE88c` (8 decimals)
 - Both use `AggregatorV3Interface` — call `latestRoundData()` and check `updatedAt` is recent ($\geq \text{heartbeat} \times 1.1$) triggers a fallback to TWAP).
-

Part 4 — Foundry Deployment Template

Deploy script (`script/00_DeployKernel.s.sol`):

```
solidity
```

```

// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.25;

import {Script, console2} from "forge-std/Script.sol";
import {Kernel, Actions} from "src/Kernel.sol";
import {IDC} from "src/IDC.sol";
import {IDCTreasury} from "src/modules/TRSRY/IDCTreasury.sol";
import {IDCMinter} from "src/modules/MINTR/IDCMinter.sol";
import {IDCPrice} from "src/modules/PRICE/IDCPrice.sol";
import {IDCRange} from "src/modules/RANGE/IDCRange.sol";
import {IDCRoles} from "src/modules/ROLES/IDCRoles.sol";
import {DEBTRegistry} from "src/modules/DEBT/DEBTRegistry.sol";
import {AaveAllocatorModule} from "src/modules/AAVEY/AaveAllocatorModule.sol";
import {HeirRegistry} from "src/modules/HEIR/HeirRegistry.sol";

contract DeployIDC is Script {
    function run() external {
        uint256 deployer = vm.envUint("DEPLOYER_PK");
        address timelock = vm.envAddress("TIMELOCK"); // future executor
        vm.startBroadcast(deployer);

        Kernel kernel = new Kernel();
        IDC idc = new IDC(address(kernel));

        IDCTreasury trsry = new IDCTreasury(kernel);
        IDCMinter mintr = new IDCMinter(kernel, address(idc));
        IDCPrice price = new IDCPrice(kernel,
            0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419, // ETH/USD
            0xF4030086522a5bEEa4988F8cA5B36dbC97BeE88c // BTC/USD
        );
        IDCRange range = new IDCRange(kernel);
        IDCRoles roles = new IDCRoles(kernel);
        DEBTRegistry debt = new DEBTRegistry(kernel);
        AaveAllocatorModule aavey = new AaveAllocatorModule(kernel,
            0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2, // Aave Pool
            0x54586bE62E3c3580375aE3723C145253060Ca0C2 // Aave Oracle
        );
        HeirRegistry heir = new HeirRegistry(kernel);

        kernel.executeAction(Actions.InstallModule, address(trsry));
        kernel.executeAction(Actions.InstallModule, address(mintr));
        kernel.executeAction(Actions.InstallModule, address(price));
        kernel.executeAction(Actions.InstallModule, address(range));
    }
}

```

```

        kernel.executeAction(Actions.InstallModule, address(roles));
        kernel.executeAction(Actions.InstallModule, address(debt));
        kernel.executeAction(Actions.InstallModule, address(aavey));
        kernel.executeAction(Actions.InstallModule, address(heir));

        // Hand off ownership to the future Timelock / Senatus contract – no EOA admin
        kernel.executeAction(Actions.ChangeExecutor, timelock);

        vm.stopBroadcast();
    }
}

```

Invariant test (`test/invariant/TreasuryInvariant.t.sol`):

```

solidity

// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.25;
import {Test} from "forge-std/Test.sol";

contract TreasuryInvariant is Test {
    // Invariant: idc.totalSupply * minBackingBps / 1e4 <=
    //             TRSRY USD + AAVEY aToken USD + DEBT expected recovery USD
    function invariant_backedSupply() public {
        uint256 lhs = idc.totalSupply() * minBackingBps / 10_000;
        uint256 rhs = trsry.reservesUSD() + aavey.aShareValueUSD() + debt.expectedRecoveryUSD();
        assertLe(lhs, rhs);
    }
}

```

CI/CD. GitHub Actions matrix on `forge test --fork-url ${ secrets.MAINNET_RPC }` at three pinned blocks; `forge coverage --report lcov`; Slither and Aderyn static analysis on PRs; Echidna invariant runs nightly; Foundry `forge fmt --check`.

Part 5 — Executive Summary and Deployment Roadmap

Phase 1 — IDC v3 core on Ethereum (months 0–3). Deploy Kernel + all 9 modules (TRSRY, MINTR, PRICE, RANGE, ROLES, DLGTE, DEPOS, DEBT, HEIR) and core RBS policies (Operator, Heart, BondCallback). Bootstrap with \$5M seed in USDS + wstETH + a small WBTC reserve. No emissions yet.

Phase 2 — AAVE V3 yield activation (months 2-4). Deploy `AaveAllocator` policy; route 60% of stables into `aEthUSDS` and `aEthDAI`. Park excess GHO in sGHO at 4.25% APR. Add Umbrella stkGHO leg only after stkGHO slashing parameters are re-confirmed at 0% per current Aave governance. Add GHO borrow line capped at 10% of TRSRY net asset value and ensure peg defense plan handles the AIP-381 9.5% APR upper bound.

Phase 3 — Cooler V3 inheritable loans (months 3-6). Port MonoCooler + LTVOracle + TreasuryBorrower; add `HEIR_` module bindings; integrate BANKON BONAFIDE verification gate for heir designation. Launch with 1% APR (favor borrowers) and a 2700 USDS/gIDC initial origination LTV (more conservative than Olympus's 2961.64 USDS/gOHM starting point).

Phase 4 — Cross-chain Algorand via Wormhole (months 4-8). Deploy Wormhole bridge wrapper + x402-AVM facilitator. mindX API monetized via x402; Parsec wallet handles AVM-side signing. Bridge IDC ↔ Algorand wIDC at the Wormhole Token Bridge endpoint `0x3ee18B2214AFF97000D974cf647E7C347E8fa585`.

Phase 5 — Full ecosystem (months 6-12). AgenticPlace agents become Convertible Deposit auction participants; mindX becomes the canonical recommender for RBS parameters with 24-hour timelock; BONAFIDE Senatus replaces the deploy multisig as Kernel executor; War Council multisig retains emergency-shutdown role only.

Risk register

Risk	Severity	Mitigation
Oracle manipulation (Chainlink heartbeat stale, TWAP gameable)	High	Dual-feed median (Chainlink × Uniswap-V3 1-hour TWAP); revert on <code>block.timestamp - updatedAt > 1.5 × heartbeat</code>
Aave V3 reserve freeze / paused asset	High	AaveAllocator policy must call <code>IPool.getReserveData()</code> and refuse deposits when reserve is paused; circuit-break to TRSRY on <code>paused == true</code>
Cooler V3 governance attack inflating LTV	Medium	LTV drip rate hard-capped in <code>LTVOracle</code> constructor (max 0.2 USDS/day, immutable); any LTV change > 5% requires 7- day timelock
HEIR fraudulent claim	Medium	Heir-claim transactions emit <code>InheritanceClaimed</code> ; 14-day delay before transfer finalizes; BANKON dispute window via Senatus
AGPL license contamination of new code	Medium	Keep all new modules in a separate Apache-2.0 package and only <i>link</i> against AGPL Kernel; ship dual-license dossier per file
Wormhole guardian collusion	High	Cap wIDC supply ≤ 5% of IDC float; require LayerZero+CCIP dual-attestation for transfers > \$1M
GHO peg break triggering AIP-381 rate ramp	Medium	Monitor 30-day average against \$0.995–\$1.005; pre-position to repay GHO borrow before rate reaches 9.5% cap

Audit recommendations. Three-phase: (1) **Code4rena or Sherlock contest** scoped to new modules (DEBT, HEIR, AAVEY) and the modified MonoCooler; (2) **Trail of Bits or Spearbit** private review of Kernel + RBS economics; (3) ongoing **Immunefi bug bounty** with \$1M critical tier funded from YRF revenue.

Recommendations

1. **Start by submoduling** `OlympusDAO/olympus-v3` **into** `lib/` **read-only and reproducing its** `forge test` **suite locally on a mainnet fork.** Until every Olympus invariant passes in your harness, do not write a single new line of IDC code. Benchmark: `forge test --match-path "test/policies/cooler/*" --fork-url $MAINNET_RPC` should be green at the pinned fork block before Phase 1 begins.
2. **Adopt Default-Framework code 1:1 except where you change semantics.** Rename only — do not refactor MINTR, TRSRY, RANGE, PRICE, ROLES, DLGTE, DEPOS. Restraint here

saves you a 6-month audit cycle.

3. **Implement DEBT, HEIR, AAVEY as the three differentiating modules**, all under Apache-2.0 in a separate `(idc-modules)` package. Treat the imported Olympus code as if it were a third-party library (because legally, it is).
 4. **Route 100% of governance through a Timelock + Senatus (BONAFIDE) executor; no EOA admin.** Use the Olympus pattern of `(kernel.executeAction(Actions.ChangeExecutor, timelockAddress))` immediately post-deploy.
 5. **Bind mindX to a 24-hour timelock for any parameter it can adjust.** Make the mindX signing key revocable via a single Senatus vote.
 6. **For Aave integration, never hard-code (Pool)** — always read via `(PoolAddressesProvider.getPool())` at the on-chain registry `(0x2f39d218133AFaB8F2B819B1066c7E434Ad94E9e)`. Aave reserves the right to upgrade the Pool proxy implementation, and the `(AaveProtocolDataProvider)` has already been upgraded once. `(Aave)`
 7. **Benchmarks that should change the plan:**
 - If stkGHO slashing rises above 0% in a future Aave proposal → remove from SAFE module composition immediately.
 - If GHO borrow rate exceeds 7% APR under AIP-381 → unwind the GHO line, rotate into sUSDS/sUSDe.
 - If MonoCooler's interest rate is raised above 1% APR by OCG → re-evaluate the Cooler V3 fork pricing.
 - If Olympus deprecates the Default Framework in favor of a Diamond/EIP-2535 architecture (rumored "V4") → re-fork from that point; do not maintain a Bophades branch indefinitely.
 8. **Bug bounty funding model:** allocate 5% of YRF weekly buyback budget to an Immunefi escrow until the bounty pool reaches \$5M, then cap.
-

Caveats

- The Olympus `(olympus-v3)` repo lacks a top-level LICENSE file; license is asserted per-file via SPDX headers. Verify every file you vendor — most are `(AGPL-3.0-or-later)` but a handful of lib imports are MIT/GPL. Apache-2.0 → AGPL is one-way only.
- Some Olympus addresses (Operator, Heart, EmissionManager) have multiple historical versions; always cross-reference the current docs page `(docs.olympusdao.finance/main/contracts/addresses)` before integrating, and confirm via `(Kernel.getModuleForKeycode(bytes5)) / Policy(isActive())` checks at the fork block.

- Aave V3 (`AaveProtocolDataProvider`) has been upgraded; the current address returned by `PoolAddressesProvider.getPoolDataProvider()` is `0x0a16f2FCC0D44FaE41cc54e079281D84A363bECD`, but the historical `0x497a1994c46d4f6C864904A9f1fac6328Cb7C8a6` is still labeled on Etherscan. Always query the provider, never hard-code.
- USDS (Sky) is the Maker/Sky DAI successor; it is not the same as DAI. Cooler V2 explicitly settles in USDS. If you bootstrap IDC in DAI, deploy a USDS migrator at launch.
- Olympus's claim that Cooler V2 is "oracle-free" refers to *liquidation* logic (no price-based liquidation), not pricing in general — the LTV Oracle still sets origination caps via governance, so an oracle-attack on the LTV Oracle would still drain the protocol; design accordingly.
- The mindX, BANKON, BONAFIDE, AgenticPlace, and PYTHAI ecosystem contracts are first-party to codephreak/PYTHAI; this report assumes their interfaces as described in the task and does not independently verify their contract source. Any production deployment should re-spec their EIP-712 signing keys, role hashes, and ROLES module entries explicitly.
- x402-avm is an actively evolving Algorand Foundation / GoPlausible spec; the v2→v2.6+ migration removed `algosdk` as a dependency in favor of `@algorandfoundation/algokit-utils@10.0.0-alpha.39`. Pin versions explicitly in `package.json`. [GitHub](#) [GitHub](#)
- The GHO borrow rate cap and sGHO 4.25% APR figures cited come from AIP-381 and the April 3, 2026 sGHO launch respectively; both are subject to ongoing governance and can change at the next AIP. Re-verify before any treasury rotation.
- Olympus has signaled "V4" research in the Medium post "Olympus: Building Better Money" but no production V4 contracts exist as of May 2026. Build IDC v3 on Bophades today; plan re-fork only if Olympus ships V4.